

PLANO DE ESTUDOS • 2026 - 2029

Roadmap de Estudos

Engenharia de software

Fundamentos, arquitetura, operação, segurança e design de API — organizados por horizonte de tempo, com critério de escolha e sinais de progresso.

Perfil: desenvolvedor de software em atividade, buscando profundidade e senioridade técnica.
Horizonte: 3 anos.
Setembro de 2026.

SEÇÃO 01

Como usar este documento

Este é um cardápio, não uma lista de tarefas. Há aqui mais material do que qualquer pessoa consome em três anos — de propósito, para você ter alternativa quando um recurso não funcionar com o seu jeito de aprender. O que vale é a regra de uso abaixo, não o tamanho das tabelas.

As cinco regras

- **No máximo três recursos ativos por vez.** Um livro, um curso ou projeto prático, uma coisa aplicada no trabalho. Terminar vale mais do que colecionar.
- **Uma hora por dia derruba quatro horas no sábado.** Profundidade se constrói por acúmulo.
- **Todo trimestre termina com algo entregue.** Um projeto no ar, um texto publicado, uma contribuição aceita, um sistema seu em produção. Sem entrega, estudo vira consumo.
- **Prática antes de teoria, sempre que der.** Leia o capítulo depois de ter batido a cabeça no problema. O conhecimento gruda no lugar onde doeu.
- **Escreva o que aprendeu.** Escrever revela o que você achou que entendeu e não entendeu.

CRITÉRIO DE SELEÇÃO

Quase tudo aqui tem meia-vida longa: rede, sistema operacional, banco de dados, arquitetura, algoritmos, operação e comunicação técnica continuam valendo daqui a vinte anos. Framework da moda ficou de fora de propósito. Aprenda o que o seu trabalho exigir, mantenha-se atualizado, e não invista sua identidade nisso: framework tem meia-vida de uns três anos, e tudo que está neste documento tem meia-vida de vinte.

SOBRE OS LINKS

Cursos mudam de endereço e canais somem. Onde tive certeza do endereço, coloquei; no resto, coloquei apenas o nome exato para você pesquisar. Se algum link não abrir, procure pelo nome: o material provavelmente continua existindo em outro lugar.

SEÇÃO 02

Horizonte 1 — Meses 1 a 12: domínio do ofício

O objetivo desta fase é deixar de ser alguém que entrega tarefa e virar alguém a quem se confia um problema. É a fase mais árida e a que mais separa quem fica raso de quem fica bom. Nada aqui depende de linguagem ou framework específico.

2.1 Uma linguagem com profundidade real

OBJETIVO	Conhecer uma linguagem a fundo: modelo de memória, concorrência, tipos, ferramentas, idiomas da comunidade. Depois disso, aprender a segunda custa um décimo do esforço.
COMO	Leia o livro de referência oficial da linguagem, não tutorial. Depois leia o código-fonte de uma biblioteca popular dela.
PRÁTICA	Exercism (exercism.org) tem mentoria humana e é gratuito. Advent of Code em dezembro, todo ano, é um ótimo ritual.
SINAL	Você sabe explicar o que o runtime faz com a sua memória e por que um trecho é lento.

2.2 Ciência da computação que você pulou

OBJETIVO	Preencher os buracos que quase todo autodidata tem: algoritmos, estruturas de dados, computação abaixo da linguagem.
ROTEIRO	Teach Yourself Computer Science (teachyourselfcs.com) — currículo curado, gratuito, com um livro e um curso por área. É o melhor mapa que existe.
LIVRO	Grokking Algorithms — Aditya Bhargava, para começar sem dor. Depois The Algorithm Design Manual — Steven Skiena.
CURSO	CS50 (cs50.harvard.edu) se a base for fraca. Algorithms , de Sedgewick (Princeton, no Coursera, gratuito para assistir). Nand2Tetris (nand2tetris.org) para construir um computador do zero.
PRÁTICA	LeetCode com moderação: útil para entrevista, limitado para engenharia. Prefira Advent of Code e projetos reais.

2.3 Sistema operacional e Linux

OBJETIVO	Modelo mental de processo, memória, arquivo, permissão e isolamento. Sem isso, comportamento estranho vira mistério.
LIVRO	Operating Systems: Three Easy Pieces (OSTEP) — gratuito online, didático e completo. Depois Computer Systems: A Programmer's Perspective .
CURSO	MIT Missing Semester (missing.csail.mit.edu) — shell, git, ferramentas. Curto e com o melhor retorno por hora desta lista inteira.
VÍDEO	Ben Eater para hardware e o que acontece no fio. Low Level para memória e binário.
PRÁTICA	OverTheWire: Bandit (overthewire.org) dá fluência de shell real, em formato de jogo.

2.4 Rede e HTTP a fundo

OBJETIVO	Explicar tudo que acontece entre digitar uma URL e a página renderizar.
LIVRO	Computer Networking: A Top-Down Approach — Kurose & Ross, como consulta. High Performance Browser Networking — Ilya Grigorik, gratuito online.
VÍDEO	Hussein Nasser — mergulhos longos em HTTP, TLS, TCP e protocolos de backend. Provavelmente o melhor canal que existe nesse recorte.
PRÁTICA	Escreva um servidor HTTP do zero, sem framework. Depois inspecione tudo no Wireshark e no DevTools.

2.5 Banco de dados

OBJETIVO	Índice, transação, isolamento, plano de execução. É onde mora a maior parte dos problemas de performance reais.
LIVRO	Designing Data-Intensive Applications — Martin Kleppmann. O livro mais importante desta lista inteira. Leia devagar, ao longo de meses.
GRÁTIS	Use The Index, Luke (use-the-index-luke.com) — o melhor material gratuito sobre índices em SQL. Curto e transformador.
CURSO	CMU 15-445 , do Andy Pavlo — aulas completas e gratuitas no YouTube, com projetos em C++.
PRÁTICA	Pegue a consulta mais lenta do sistema que você mantém e otimize com EXPLAIN na mão. Escreva o que descobriu.

2.6 Testes, refatoração e código legado

OBJETIVO	Construir a rede de segurança que permite mudar código com confiança. Com IA gerando volume, isso virou a habilidade central, não acessória.
LIVRO	Working Effectively with Legacy Code — Michael Feathers. Refactoring — Martin Fowler. Tidy First? — Kent Beck (curto). Test-Driven Development by Example — Kent Beck.
VÍDEO	Continuous Delivery , do Dave Farley — canal excelente e subestimado. CodeAesthetic para vídeos curtos sobre qualidade de código.
PRÁTICA	Pegue um módulo sem teste no trabalho, cubra por comportamento (não por implementação) e refatore por dentro.

2.7 Disciplina com ferramentas de IA

OBJETIVO	Aproveitar a velocidade sem perder a competência. Este é o risco real e já mensurado desta década.
REGRA 1	Tente primeiro, compare depois. A parte difícil é onde o aprendizado acontece.
REGRA 2	Nunca aceite código que você não consegue explicar em voz alta.
REGRA 3	Meça. Em experimento controlado, desenvolvedores experientes ficaram mais lentos usando IA enquanto se sentiam mais rápidos. Percepção não é dado.
LEITURA	Relatórios DORA (dora.dev) e a Stack Overflow Developer Survey (survey.stackoverflow.co). Gratuitos e metodologicamente sérios.

SEÇÃO 03

Horizonte 2 — Anos 2 e 3: projetar e operar

Aqui você sai da execução e entra na decisão. É exatamente a fronteira que os dados de mercado mostram: a função de transcrever requisito em código encolheu muito; a de projetar, decidir e responder pelo sistema praticamente não se mexeu.

3.1 Arquitetura e design de software

OBJETIVO	Saber onde traçar fronteira, o que separar, o que juntar — e registrar por escrito o porquê.
LIVRO	A Philosophy of Software Design — John Ousterhout (curto, excelente). Fundamentals of Software Architecture — Richards & Ford. Software Architecture: The Hard Parts para trade-offs difíceis.
GRÁTIS	Software Engineering at Google — abseil.io/resources/swe-book . Trata de engenharia no tempo: manutenção, escala de equipe, cultura.
VÍDEO	ByteByteGo e Gaurav Sen para system design. Palestras do GOTO Conferences e do arquivo do Strange Loop .
PRÁTICA	Escreva um ADR (Architecture Decision Record) para cada decisão relevante. Duas páginas: contexto, opções, escolha, consequência.
SINAL	Alguém usa um ADR seu para decidir algo sem precisar te consultar.

3.2 Modelagem de domínio

OBJETIVO	Traduzir negócio em modelo. É a parte do trabalho que mais depende de contexto humano e menos pode ser delegada.
LIVRO	Domain-Driven Design Distilled — Vaughn Vernon (comece por aqui, não pelo livro azul). Patterns of Enterprise Application Architecture — Fowler.
VÍDEO	Canal Full Cycle , em português, tem material sólido de DDD e arquitetura aplicada.
PRÁTICA	Modele um subdomínio do seu trabalho com linguagem ubíqua e mostre para alguém da área de negócio. Se a pessoa não reconhecer os termos, o modelo está errado.

3.3 Sistemas distribuídos

OBJETIVO	Consistência, falha parcial, relógio, consenso, idempotência. Não para construir do zero, e sim para não ser surpreendido.
LIVRO	Designing Data-Intensive Applications de novo, agora os capítulos finais. Database Internals — Alex Petrov. Building Microservices — Sam Newman, com ceticismo saudável.
CURSO	MIT 6.5840 (antigo 6.824) — aulas e laboratórios públicos, em Go. Dos melhores que existem; exige dedicação real.
PRÁTICA	Gossip Glomers (fly.io/dist-sys) — desafios com verificação automática, gratuitos.
SINAL	Você explica por que um retry ingênuo pode duplicar uma cobrança.

3.4 Performance e diagnóstico

OBJETIVO	Descobrir por que o sistema está lento sem chutar. Habilidade rara e imediatamente visível para quem está em volta.
LIVRO	Systems Performance — Brendan Gregg. Referência absoluta. BPF Performance Tools , do mesmo autor, para ir mais fundo.
GRÁTIS	O site brendangregg.com tem boa parte do método publicado, incluindo o USE Method e os flame graphs.
PRÁTICA	Faça profiling de um serviço real. Meça antes, mude uma coisa só, meça depois. Escreva o resultado, mesmo quando der errado.

3.5 Operação, observabilidade e entrega

OBJETIVO	O gargalo se deslocou para depois do commit. Mais código entrando mais rápido sobrecarrega pipeline e revisão — e é aí que a instabilidade aparece.
LIVRO	Site Reliability Engineering e The SRE Workbook — Google, gratuitos em sre.google/books . Release It! — Michael Nygard. Accelerate — Forsgren, Humble e Kim. Observability Engineering — Majors, Fong-Jones e Miranda.
CURSO	Material gratuito da Grafana e da Honeycomb sobre observabilidade.
VÍDEO	Palestras da SREcon no YouTube. Canal Continuous Delivery , do Dave Farley, para entrega contínua.
PRÁTICA	Instrumente um serviço com log estruturado, métrica e trace. Provoque uma falha de propósito e diagnostique só pelos sinais.
SINAL	Você fica de plantão sem pânico, porque sabe onde olhar.

3.6 Cloud e infraestrutura

OBJETIVO	Uma nuvem só, com profundidade: IAM, rede, custo, isolamento.
CURSO	AWS Skill Builder ou Google Cloud Skills Boost — trilhas oficiais, boa parte gratuita. KodeKloud para Kubernetes com laboratório.
PRÁTICA	Kubernetes the Hard Way , de Kelsey Hightower (gratuito no GitHub). Doloroso e insubstituível.
ATENÇÃO	Saber quanto custa o que você desenhou é raro e muito valorizado. Aprenda a ler fatura de nuvem e a estimar custo antes de construir.

SEÇÃO 04

Horizonte 3 — Multiplicar

O que separa sênior de alguém realmente difícil de substituir não é mais conhecimento técnico: é julgamento, contexto e alcance. Esta seção é curta porque quase nada aqui se aprende lendo — mas os livros ajudam a nomear o que você já está vivendo.

4.1 Escrita técnica

POR QUÊ	Design doc, RFC e postmortem são como sua influência escala além do que você digita. Num mundo onde gerar texto ficou barato, clareza de pensamento ficou mais valiosa, não menos.
LIVRO	On Writing Well — William Zinsser. The Sense of Style — Steven Pinker.
CURSO	Google Technical Writing — gratuito, curto, subestimado.
PRÁTICA	Escreva o postmortem do próximo incidente da sua equipe, mesmo sem ninguém pedir. Sem culpado: causa, contribuintes e ação.

4.2 Senioridade e alcance

LIVRO	The Staff Engineer's Path — Tanya Reilly. Staff Engineer — Will Larson. The Effective Engineer — Edmond Lau. Team Topologies — Skelton & Pais.
CLÁSSICOS	The Mythical Man-Month — Fred Brooks. Peopleware — DeMarco & Lister. Antigos e ainda certos sobre quase tudo.
PRÁTICA	Pegue o problema feio que ninguém quer: o legado, o incidente recorrente, a dívida que todos reclamam. É onde a confiança se constrói rápido.

4.3 Entender o negócio

POR QUÊ	Saber por que a empresa ganha dinheiro é o que permite decidir bem sem perguntar. É também exatamente o contexto que nenhuma ferramenta tem.
COMO	Converse com vendas, suporte e financeiro. Peça para assistir a uma call com cliente. Entenda o custo por transação do que você mantém.
SINAL	Você já disse 'isso não resolve o problema' ou 'a versão simples atende' e teve razão. Construir perfeitamente a coisa errada é o erro mais caro que existe.

SOBRE A IDEIA DE SER INDISPENSÁVEL

Vale dizer o que ficou de fora de propósito. Quem se torna indispensável costuma virar a pessoa que não pode sair de férias, não é promovida e não consegue trocar de time — presa por ser a única que entende aquele sistema. O objetivo melhor é ser difícil de substituir por um bom motivo: você resolve problemas que poucos resolvem, e leva essa capacidade com você para onde quiser ir.

SEÇÃO 05

Secure Software Engineering

Segurança não é uma disciplina ao lado da engenharia: é uma propriedade do código que você já escreve. Este eixo atravessa os três horizontes. No H1 é o que todo desenvolvedor precisa saber para não abrir buraco; no H2 é construir com segurança por padrão; no H3 é tratar segurança como propriedade da arquitetura.

UMA NOTA SOBRE VOLUME DE CÓDIGO

Com IA gerando muito mais código e a revisão humana no mesmo ritmo de antes, a superfície de ataque cresce mais rápido que a defesa. A falha típica não está no algoritmo criptográfico: está em autorização por objeto, configuração de nuvem e dependência. São exatamente os pontos que uma ferramenta de geração não verifica por você.

Horizonte 1 — o mínimo que ninguém pode não saber

5.1 HTTP security

ESCOPO	Same-origin policy e CORS, cabeçalhos de segurança (CSP, HSTS, X-Content-Type-Options, Referrer-Policy), cookies com Secure, HttpOnly e SameSite, TLS e redirecionamento, clickjacking.
RECURSO	MDN Web Security e o OWASP Secure Headers Project , ambos gratuitos. Para o porquê das coisas, The Tangled Web — Michal Zalewski.
PRÁTICA	Rode um scanner de cabeçalho no seu próprio serviço e corrija o que aparecer. Depois entenda por que cada cabeçalho existe.

5.2 Autenticação e sessão

ESCOPO	Hash de senha (argon2 ou bcrypt, nunca hash simples), sessão por cookie versus token, expiração e rotação, logout que realmente invalida, MFA, e o ponto mais atacado de todos: recuperação de conta.
LIVRO	Alice and Bob Learn Application Security — Tanya Janca. A melhor porta de entrada para quem vem de desenvolvimento.
CURSO	PortSwigger Web Security Academy (portswigger.net/web-security), trilhas de authentication e session management. Gratuito e com laboratório.
ARMADILHA	Autenticação você quase nunca deve implementar do zero. Mas precisa entender o suficiente para avaliar a biblioteca que vai usar.

5.3 Injeção e validação de entrada

ESCOPO	SQL injection e consulta parametrizada, command injection, path traversal, XSS com codificação por contexto de saída, deserialização insegura, SSRF.
PRINCÍPIO	Valide na entrada, codifique na saída, e nunca construa comando ou consulta por concatenação. ORM reduz o risco, não elimina: query bruta e cláusula dinâmica continuam vulneráveis.
CURSO	PortSwigger Academy, trilhas de SQL injection, XSS e SSRF. OWASP Juice Shop para praticar localmente.
LIVRO	Web Security for Developers — Malcolm McDonald. Curto e direto ao ponto.

5.4 OWASP básico

TOP 10	Serve para conhecer as classes de falha. Não decore a lista: entenda a mecânica de cada uma no código.
CHEAT SHEETS	A OWASP Cheat Sheet Series é o material mais útil do projeto e o menos conhecido. Responde 'como faço certo' em uma página por assunto.
ASVS	Padrão de verificação usado em trabalho sério. Vale conhecer cedo: dá vocabulário para discutir requisito de segurança com precisão.
ONDE	Tudo em owasp.org , gratuito.
SINAL	Você revisa um pull request e enxerga a falha antes de qualquer ferramenta.

Horizonte 2 — construir com segurança por padrão

5.5 Threat modeling

ESCOPO	STRIDE, diagrama de fluxo de dados, fronteiras de confiança. Feito no desenho, não depois do incidente.
LIVRO	Threat Modeling: Designing for Security — Adam Shostack.
FERRAMENTA	OWASP Threat Dragon , gratuito. Um quadro branco também serve.
PRÁTICA	Modele uma feature que você vai construir no próximo mês. Quatro perguntas: o que estamos construindo, o que pode dar errado, o que vamos fazer sobre isso, fizemos um bom trabalho?

5.6 OAuth 2.0 e OpenID Connect

ESCOPO	A diferença entre autenticação e autorização, authorization code com PKCE, tokens de acesso e de identidade, escopo, refresh, audiência. E os fluxos que morreram: implicit e password grant.
RECURSO	oauth.net e o OAuth 2 Simplified , de Aaron Parecki. A palestra <i>OAuth 2.0 and OpenID Connect in plain English</i> , de Nate Barbettini, no canal OktaDev , é a melhor explicação em vídeo.
ARMADILHA	Quase todo bug grave de OAuth vem de validação frouxa: <code>redirect_uri</code> , <code>state</code> , audiência do token, assinatura do JWT. Leia a especificação de boas práticas de segurança, não só o fluxo feliz.

5.7 API security

ESCOPO	OWASP API Security Top 10 . O campeão absoluto é BOLA (autorização quebrada por objeto, também chamado IDOR): a API confere quem você é e esquece de conferir se aquele recurso é seu.
TAMBÉM	Mass assignment, exposição excessiva de dados na resposta, falta de limite de consumo, autorização por função em vez de por recurso.
LIVRO	API Security in Action — Neil Madden. O melhor livro do assunto, e cobre também token, escopo e mTLS.
PRÁTICA	Pegue um endpoint seu e tente acessar o recurso de outro usuário trocando só o identificador. Se funcionar, você achou o bug mais comum da internet.

5.8 SAST, SCA e DAST na esteira

O QUE CADA UM FAZ	SAST lê seu código em busca de padrão vulnerável. SCA olha suas dependências e compara com bases de CVE. DAST ataca a aplicação rodando. Os três pegam coisas diferentes e nenhum substitui revisão.
FERRAMENTAS	Semgrep e CodeQL para SAST, Trivy e Dependabot ou Renovate para dependência, OWASP ZAP para DAST. Todos com versão gratuita.
REALIDADE	O problema dessas ferramentas é ruído. Comece bloqueando só o que é grave e comprovado; alerta que todo mundo ignora é pior que nenhum alerta.

5.9 Secrets

REGRA	Segredo nunca no repositório, nunca em variável de build exposta, nunca em log. Use cofre: Vault , ou o gerenciador de segredos da sua nuvem.
ROTAÇÃO	Segredo que nunca roda é segredo vazado esperando a hora. Prefira credencial de curta duração a chave estática.
DETECÇÃO	gitleaks e trufflehog no pré-commit e na esteira. Vale rodar no histórico do repositório uma vez, pelo susto.

5.10 Supply chain

ESCOPO	Lockfile e versão fixada, SBOM, assinatura e proveniência de artefato, typosquatting, dependência transitiva, script de pós-instalação.
RECURSO	SLSA (slsa.dev) para níveis de proveniência, Sigstore e cosign para assinatura, OpenSSF Scorecard para avaliar a saúde de um projeto antes de adotá-lo. Tudo gratuito.
PRÁTICA	Gere o SBOM do seu projeto e olhe de verdade a lista. A quantidade de código de terceiros que você está executando costuma ser um choque.

Horizonte 3 — segurança como propriedade da arquitetura

5.11 Cloud security e IAM

POR QUÊ	É onde acontece a maior parte dos incidentes reais. Não é código vulnerável: é permissão ampla demais e recurso exposto por configuração.
IAM	Menor privilégio de verdade, papel em vez de usuário, identidade de workload (federação OIDC na esteira em vez de chave de longa duração), separação de ambiente, revisão periódica de permissão.
CLOUD	Isolamento de rede, criptografia em trânsito e em repouso, log de auditoria ligado e retido, avaliação de postura. Aprenda o modelo de responsabilidade compartilhada da sua nuvem antes de qualquer ferramenta.
PRÁTICA	Audite as permissões do seu próprio projeto. Quase sempre há uma credencial com muito mais poder do que precisa.

5.12 Runtime security

ESCOPO	Hardening de container (usuário não root, sistema de arquivos somente leitura, capacidades mínimas), política de admissão, isolamento de workload, detecção em execução.
FERRAMENTA	Falco para detecção baseada em eBPF, Trivy para imagem, políticas de admissão no Kubernetes.
LIVRO	Container Security — Liz Rice. Explica o que realmente isola um container, que é menos do que a maioria imagina.

5.13 Security architecture

ESCOPO	Fronteiras de confiança, zero trust aplicado com juízo, multi-tenancy e isolamento de dados, criptografia de dado sensível, retenção e minimização, LGPD, plano de resposta a incidente.
POSTURA	A pergunta de arquitetura não é 'como impedimos a invasão', é 'quando um componente for comprometido, o que o atacante alcança a partir dali'. Desenhe para conter o estrago, não só para evitá-lo.
SINAL	Você consegue apontar, no diagrama do sistema, qual componente comprometido causaria o maior dano — e por quê.

SEÇÃO 06

Design de API, integração e resiliência

Esta é a parte da engenharia que aparece em toda vaga sênior e quase nunca em curso. São as decisões de contrato e de comportamento sob falha: o que envelhece bem, o que quebra cliente, o que derruba o sistema às três da manhã.

6.1 Estilos: REST, GraphQL e gRPC

REST	Padrão para superfície pública. Ganha em cache HTTP, ferramental, familiaridade e depuração. Perde quando o cliente precisa de composições muito variadas de dados.
GRAPHQL	Bom quando há muitos consumidores diferentes e o over-fetching dói de verdade. O custo é real: autorização fica por campo, o problema N+1 aparece sempre, e limitar consumo exige análise de custo da consulta.
GRPC	Serviço para serviço, contrato forte, streaming, baixa latência. Ruim como API pública de navegador. Protobuf força você a pensar em evolução de esquema, o que é uma vantagem disfarçada de chatice.
LIVRO	API Design Patterns — JJ Geewax (Google). O melhor livro do assunto.
GRÁTIS	Google AIP (aip.dev) e as diretrizes de API da Microsoft no GitHub. Production Ready GraphQL — Marc-André Giroux, se for por esse caminho.

6.2 Autenticação e autorização na API

DISTINÇÃO	Autenticação responde quem é você; autorização responde se você pode fazer isto com este recurso. A maior parte das falhas graves está na segunda, não na primeira.
PADRÕES	Token de curta duração com escopo, verificação de audiência, autorização por objeto em cada acesso (não só no início da requisição), mTLS ou token de serviço para tráfego interno.
VER TAMBÉM	Seções 5.6 e 5.7. Design de API e segurança de API são o mesmo assunto visto de dois ângulos.

6.3 Contratos que envelhecem bem

IDEMPOTÊNCIA	Chave de idempotência em toda operação de escrita que o cliente pode repetir. Exactly-once na rede é ilusão: o que existe é at-least-once com consumidor idempotente. As docs da Stripe são o exemplo canônico e valem uma leitura atenta.
PAGINAÇÃO	Cursor em vez de offset. Offset fica lento e pula ou duplica registro quando os dados mudam entre páginas. Defina limite máximo e devolva sempre o cursor seguinte.
VERSÕES	Evolução compatível é quase sempre melhor do que versão nova: adicione campo, nunca mude o significado de um existente. Quando versionar for inevitável, publique prazo de depreciação e meça quem ainda usa a versão velha.
RATE LIMITING	Token bucket por chave de cliente, cabeçalhos informando o limite restante, resposta 429 com Retry-After. É controle de custo, de estabilidade e de segurança ao mesmo tempo.

6.4 Resiliência: timeouts, retries e circuit breaker

TIMEOUT	Toda chamada de rede tem timeout. Sem exceção. Chamada sem timeout é vazamento de recurso esperando a vez.
RETRY	Só em erro transiente e só em operação idempotente. Com backoff exponencial e jitter — sem jitter, todos os clientes voltam juntos e você inventou um ataque contra si mesmo. Limite o total de tentativas na cadeia inteira.
CIRCUIT BREAKER	Para de bater em serviço caído e dá tempo dele voltar. Junto com bulkhead (isolar pools), load shedding e degradação graciosa.
LIVRO	Release It! — Michael Nygard. É literalmente o livro deste bloco, e cada capítulo é uma história real de queda.

6.5 Caching

CAMADAS	Cliente, CDN, aplicação, banco. Decida conscientemente em qual você está mexendo — cache em camada errada resolve pouco e confunde muito.
HTTP	ETag, Cache-Control, revalidação condicional. É cache que você ganha de graça e a maioria dos times ignora.
PADRÕES	Cache-aside é o padrão; write-through e write-behind têm lugar. TTL sempre, invalidação explícita quando der.
ARMADILHAS	Stampede na expiração (resolva com single-flight ou TTL com ruído), dado obsoleto onde não pode haver, e cache virando fonte de verdade sem ninguém perceber.

6.6 Filas, eventos e webhooks

FILA X STREAM	Fila distribui trabalho e some com a mensagem; stream mantém o log e permite reprocessar. Escolher errado custa caro depois.
ENTREGA	At-least-once é o normal: o consumidor precisa ser idempotente. Dead letter queue obrigatória, com alguém olhando. Ordenação só dentro de partição, e ela custa paralelismo.
OUTBOX	Para publicar evento e gravar no banco atomicamente. Padrão simples que resolve uma classe inteira de inconsistência.
WEBHOOKS	Assinatura HMAC no corpo, timestamp contra replay, retry com backoff, idempotência no receptor. E cuidado com SSRF: URL de destino fornecida pelo usuário é entrada hostil.
LIVRO	Enterprise Integration Patterns — Hohpe & Woolf, como referência. Kleppmann, capítulo sobre processamento de stream, para os fundamentos.

6.7 CI/CD

PRINCÍPIOS	Build reprodutível, artefato imutável construído uma vez e promovido entre ambientes (nunca reconstruído), testes em estágios do mais rápido ao mais lento, trunk-based com lote pequeno.
DEPLOY	Progressivo: canário ou blue-green, feature flag para separar deploy de release, rollback automático baseado em sinal. Deploy grande e raro é a causa, não a solução, de instabilidade.
SEGURANÇA DA ESTEIRA	Permissão mínima no runner, federação OIDC em vez de chave estática, segredo vindo de cofre, assinatura do artefato. A esteira tem acesso a produção: trate como alvo de primeira linha.
LIVRO	Continuous Delivery — Humble & Farley para a base conceitual. Accelerate para a evidência de que isso funciona.
MÉTRICA	As quatro do DORA: frequência de deploy, lead time, taxa de falha de mudança e tempo de recuperação. Meça antes de tentar melhorar.

SEÇÃO 07

Trilhas por especialidade

As seções anteriores valem para qualquer desenvolvedor. Esta é o recorte por área. Escolha uma como principal e mantenha alfabetização nas outras — o perfil mais valorizado hoje é o de quem tem uma profundidade e várias competências vizinhas.

Backend e sistemas

LIVRO	Kleppmann, Database Internals , Release It! , Systems Performance (Gregg).
CURSO	CMU 15-445 (banco), MIT 6.5840 (distribuídos), Gossip Glomers.
VÍDEO	Hussein Nasser , ByteByteGo , CMU Database Group , Jon Gjengset (Rust e sistemas, em sessões longas).
PROJETO	Um banco chave-valor com write-ahead log. Um servidor HTTP do zero. Um pool de conexões. Nessa ordem.

Frontend e web

CURSO	Full Stack Open (fullstackopen.com), da Universidade de Helsinque — gratuito e sério. The Odin Project (theodinproject.com) para base ampla. web.dev , do Google, para performance e padrões.
LIVRO	Refactoring UI — Wathan & Schoger, para quem não vem de design. Inclusive Components — Heydon Pickering, para acessibilidade.
VÍDEO	Kevin Powell (o melhor canal de CSS que existe), Web Dev Simplified , Jack Herrington , Theo , Fireship para panorama rápido.
FOCO	Performance real (Core Web Vitals), acessibilidade e estado. São os três assuntos que separam quem monta tela de quem constrói produto.

Plataforma, DevOps e SRE

LIVRO	Site Reliability Engineering (gratuito), The SRE Workbook , Continuous Delivery — Humble & Farley, Team Topologies .
CURSO	KodeKloud, Kubernetes the Hard Way, trilhas oficiais de nuvem.
VÍDEO	TechWorld with Nana , NetworkChuck , palestras da SREcon .
PROJETO	Monte uma esteira completa do zero: build, teste, deploy, rollback, observabilidade e alerta. Documente o custo mensal.

Dados e IA aplicada

LIVRO	Kleppmann para fundamentos de dados. Fundamentals of Data Engineering — Reis & Housley. Designing Machine Learning Systems — Chip Huyen.
CURSO	Neural Networks: Zero to Hero , do Andrej Karpathy — gratuito no YouTube, constrói tudo do zero. Melhor material introdutório que existe.
FOCO	Se o interesse for IA aplicada a produto, o gargalo raramente é o modelo: é dado, avaliação e integração. Estude avaliação de sistema com mais cuidado do que arquitetura de rede neural.

SEÇÃO 08

Livros, por ordem de prioridade

Se você só puder ler quatro livros nos próximos três anos, leia os quatro primeiros. A coluna *quando* indica a fase em que o livro rende mais.

Título	Autor	Quando	Por quê
Designing Data-Intensive Applications	Martin Kleppmann	H1-H2	O mais importante da lista. Muda como você pensa sobre dados, falha e consistência.
A Philosophy of Software Design	John Ousterhout	H1-H2	Curto e direto, sobre complexidade e abstração. Retorno por página altíssimo.
The Pragmatic Programmer (20 anos)	Hunt & Thomas	H1	Postura profissional e hábitos. O livro que mais gente deveria reler.
Working Effectively with Legacy Code	Michael Feathers	H1	Como mudar código que você não escreveu sem quebrar. Mais relevante do que nunca.
Operating Systems: Three Easy Pieces	Arpaci-Dusseau (grátis)	H1	Sistema operacional explicado com clareza rara. Gratuito e completo.
Computer Systems: A Programmer's Perspective	Bryant & O'Hallaron	H1	O que acontece abaixo da sua linguagem. Denso e definitivo.
Refactoring (2ª ed.)	Martin Fowler	H1	Catálogo de transformações seguras. Livro de mesa, não de leitura corrida.
Test-Driven Development by Example	Kent Beck	H1	Curto. Mesmo que você não adote TDD, muda como você pensa em passos pequenos.
The Algorithm Design Manual	Steven Skiena	H1	Algoritmos com foco em aplicar, não em provar. Melhor que CLRS para praticante.
Release It! (2ª ed.)	Michael Nygard	H2	Padrões de estabilidade em produção. Cada capítulo é uma falha real.
Accelerate	Forsgren, Humble & Kim	H2	A pesquisa por trás das métricas DORA. Curto e baseado em dado.
Site Reliability Engineering	Google (grátis)	H2	Como operar sistema grande. Leia por capítulo, conforme a necessidade.
Software Engineering at Google	Winters et al. (grátis)	H2-H3	Engenharia no tempo: manutenção, escala de equipe, cultura de código.
Fundamentals of Software Architecture	Richards & Ford	H2	Panorama de estilos e trade-offs. Bom mapa geral da disciplina.
Domain-Driven Design Distilled	Vaughn Vernon	H2	A versão curta e legível do DDD. Comece aqui, não pelo livro azul.
Database Internals	Alex Petrov	H2	Como bancos funcionam por dentro. Para depois do Kleppmann.

Título	Autor	Quando	Por quê
Systems Performance (2ª ed.)	Brendan Gregg	H2	Referência de diagnóstico e performance. Consulta obrigatória para backend.
Observability Engineering	Majors, Fong-Jones & Miranda	H2	Por que monitoramento tradicional não basta em sistema distribuído.
Continuous Delivery	Humble & Farley	H2	A base conceitual de esteira e deploy. Envelheceu bem.
Team Topologies	Skelton & Pais	H2-H3	Como o desenho da equipe determina o desenho do sistema. Curto e útil.
The Staff Engineer's Path	Tanya Reilly	H3	O que muda quando o trabalho deixa de ser escrever código.
The Effective Engineer	Edmond Lau	H3	Sobre alavancagem: onde o esforço rende mais. Prático.
The Mythical Man-Month	Fred Brooks	H3	De 1975 e ainda certo sobre quase tudo que dá errado em projeto de software.
Crafting Interpreters	Robert Nystrom (grátis)	Projeto	Construa uma linguagem do zero, duas vezes. Grátis e magistral.
Alice and Bob Learn Application Security	Tanya Janca	Sec H1	A melhor porta de entrada em segurança para quem vem de desenvolvimento.
Web Security for Developers	Malcolm McDonald	Sec H1	Curto e direto. As falhas mais comuns explicadas no nível do código.
The Tangled Web	Michal Zalewski	Sec H1	Por que a web é insegura por construção. Denso e brilhante.
Threat Modeling	Adam Shostack	Sec H2	Como pensar em ataque durante o projeto, não depois do incidente.
API Security in Action	Neil Madden	Sec H2	O melhor livro sobre segurança de API. Cobre token, escopo, mTLS e autorização.
API Design Patterns	JJ Geewax	H2	O melhor livro sobre desenho de API. Paginação, versionamento, operações longas.
Enterprise Integration Patterns	Hohpe & Woolf	H2	Catálogo de mensageria e integração. Referência, não leitura corrida.
Container Security	Liz Rice	Sec H3	O que realmente isola um container — que é menos do que a maioria imagina.

SEÇÃO 09

Cursos, plataformas e prática

Recurso	Custo	Onde	Para quê
Teach Yourself Computer Science	Grátis	teachyourselfcs.com	Currículo curado de CS com um livro e um curso por área. O melhor mapa que existe.
MIT Missing Semester	Grátis	missing.csail.mit.edu	Shell, git, ferramentas. Curto e com o melhor retorno por hora desta lista.
CS50	Grátis	cs50.harvard.edu	Introdução ampla de ciência da computação. Produção excelente.
Full Stack Open	Grátis	fullstackopen.com	Curso de web moderno da Universidade de Helsinque. Sério e com certificado.
The Odin Project	Grátis	theodinproject.com	Trilha completa de web, orientada a projeto e comunidade.
Nand2Tetris	Grátis	nand2tetris.org	Construir um computador do transistor ao sistema operacional. Transformador.
Crafting Interpreters	Grátis	craftinginterpreters.com	Livro-projeto: escreva uma linguagem do zero, duas vezes.
Use The Index, Luke	Grátis	use-the-index-luke.com	O melhor material gratuito sobre índices em SQL. Curto e imediato.
CMU 15-445 Database Systems	Grátis	YouTube / site do curso	Aulas do Andy Pavlo mais projetos. Melhor curso público de banco de dados.
MIT 6.5840 Distributed Systems	Grátis	Site do MIT PDOS	Laboratórios sérios em Go. Exige comprometimento real.
Gossip Glomers	Grátis	fly.io/dist-sys	Desafios de sistemas distribuídos com verificação automática.
Kubernetes the Hard Way	Grátis	GitHub / Kelsey Hightower	Montar um cluster do zero, na mão. Doloroso e insubstituível.
Build Your Own X	Grátis	GitHub (codecrafters-io)	Coletânea de tutoriais para reconstruir ferramentas conhecidas do zero.
Neural Networks: Zero to Hero	Grátis	YouTube / Karpathy	Fundamentos de redes neurais construídos do zero, linha por linha.
Google Technical Writing	Grátis	Google Developers	Curso curto de escrita técnica. Muito subestimado.
Exercism	Grátis	exercism.org	Exercícios com mentoria humana. Bom para linguagem nova com feedback real.
Advent of Code	Grátis	adventofcode.com	Ritual anual de dezembro. Ótimo para exercitar algoritmo sem clima de entrevista.

Recurso	Custo	Onde	Para quê
CodeCrafters	Pago	codecrafters.io	Reconstruir Redis, git, um shell e outros, com testes automáticos guiando.
Frontend Masters	Pago	frontendmasters.com	Cursos longos com instrutores de primeira linha. Bom além do frontend.
KodeKloud	Pago	kodekloud.com	Kubernetes e DevOps com laboratório no navegador.
Full Cycle	Pago	Brasil	Arquitetura, DDD, Go e sistemas, em português e com profundidade.
PortSwigger Web Security Academy	Grátis	portswigger.net/web-security	Laboratórios de segurança web com explicação. Melhor que a maioria dos cursos pagos.
OWASP: Top 10, Cheat Sheets, ASVS	Grátis	owasp.org	Classes de falha, o como-fazer-certo por assunto e o padrão de verificação.
OWASP Juice Shop	Grátis	owasp.org	Aplicação propositalmente vulnerável para treinar localmente.
OAuth 2 Simplified	Grátis	oauth.net / Aaron Parecki	A explicação mais clara de OAuth e OIDC, sem atalho nem simplificação errada.
Google AIP	Grátis	aip.dev	Diretrizes de desenho de API usadas internamente pelo Google. Concretas e discutidas.
SLSA / Sigstore / OpenSSF Scorecard	Grátis	slsa.dev e afins	Proveniência, assinatura de artefato e avaliação de saúde de dependência.
Documentação da API da Stripe	Grátis	docs da Stripe	Referência de fato em idempotência, paginação e versionamento. Leia como estudo.

SEÇÃO 10

Canais do YouTube

Vídeo é ótimo para ganhar intuição e péssimo para ganhar profundidade. Use como porta de entrada e como companhia enquanto pratica, nunca como substituto de livro ou projeto. A lista está separada por finalidade.

Fundamentos e sistemas

Canal	Foco e por que assistir
Ben Eater	Hardware, rede e o que acontece no fio. Constrói tudo do zero na bancada. Didática excepcional.
Hussein Nasser	Backend, protocolos e banco de dados. Mergulhos longos em HTTP, TLS, TCP e engines. Provavelmente o melhor canal de backend que existe.
Computerphile	Ciência da computação explicada por pesquisadores. Ótimo para conceitos isolados.
CMU Database Group	Curso completo de banco de dados, nível universitário, com o Andy Pavlo.
Jon Gjengset	Rust e sistemas em sessões longas e sem edição. Para ver alguém pensando em tempo real.
Low Level	Memória, binário e o que a linguagem esconde de você.
Andrej Karpathy	Redes neurais construídas do zero. Melhor material introdutório de IA que existe, e é gratuito.
MIT OpenCourseWare	Cursos universitários completos e gratuitos.

Arquitetura, qualidade e carreira

Canal	Foco e por que assistir
Continuous Delivery	Dave Farley. Entrega contínua, testes, qualidade e crítica bem fundamentada de modismo. Subestimado.
ByteByteGo	System design em formato curto e visual. Bom para vocabulário.
Gaurav Sen	System design em formato longo, com trade-off explícito.
CodeAesthetic	Vídeos curtos e afiados sobre qualidade de código. Poucos vídeos, todos bons.
GOTO Conferences	Arquivo enorme de palestras de conferência. Procure por nome de autor: quase todos os que estão na seção de livros têm palestra ali.
InfoQ / QCon	Casos reais de engenharia em empresas grandes.
ThePrimeagen	Fluxo de trabalho, editor e opinião forte sobre a indústria. Filtre o entretenimento do conteúdo.
Dreams of Code	Ferramentas e práticas de desenvolvimento, bem produzido.

Canal	Foco e por que assistir
OktaDev	Identidade, OAuth 2.0 e OpenID Connect. A palestra do Nate Barbettini sobre OAuth em inglês simples é a melhor explicação em vídeo do assunto.
OWASP	Canal oficial. Palestras de AppSec, threat modeling e segurança de API, com gente que escreve os padrões.
Confluent	Eventos, streaming e Kafka. Bom mesmo que você não use Kafka: os conceitos de log, partição e ordenação são gerais.

Frontend e web

Canal	Foco e por que assistir
Kevin Powell	CSS. O melhor canal que existe no assunto, sem exagero.
Web Dev Simplified	JavaScript e web moderna, explicação clara e direta.
Jack Herrington	React e arquitetura de frontend, com nível técnico alto.
Fireship	Panorama rápido de tecnologia. Bom para saber que algo existe, ruim para aprender.
Theo	Debate sobre ecossistema web. Opinativo; útil para acompanhar o que a comunidade está discutindo.

Em português

Canal	Foco e por que assistir
Fabio Akita	Carreira, fundamentos e crítica de hype. Vídeos longos e densos; excelente para calibrar expectativa sobre a indústria.
Full Cycle	Wesley Willians. Arquitetura, DDD, Go, sistemas distribuídos. Conteúdo técnico sério em português.
Rodrigo Branas	Arquitetura, testes e clean code, com prática em código.
Lucas Montano	Carreira e bastidores da indústria, formato leve.
Código Fonte TV	Panorama e notícia da área. Bom para acompanhar.
Filipe Deschamps	Mais introdutório, com bons projetos guiados do início ao fim.
Rocketseat	Web e mobile, formato de bootcamp. Útil na base.

Podcast em português que vale acompanhar: Hipsters Ponto Tech. Eventos e comunidade no Brasil: The Developer's Conference (TDC), QCon São Paulo e os meetups locais.

SEÇÃO 11

Projetos que ensinam, e os primeiros 90 dias

Curso ensina a reconhecer; projeto ensina a fazer. Os projetos abaixo foram escolhidos porque cada um força um conceito difícil que não dá para contornar. Reconstruir uma ferramenta que você usa todo dia é a forma mais rápida de entender o que ela resolve.

Projeto	Fase	O que ele força você a aprender
Servidor HTTP do zero	Início	Socket, parsing, keep-alive, concorrência. Você nunca mais lê documentação de framework web do mesmo jeito.
Shell simples	Início	Processo, fork, pipe, redirecionamento, sinal. Sistema operacional deixa de ser abstrato.
Interpretador de linguagem	Meio	Siga o Crafting Interpreters. Ensina parsing, árvore sintática, escopo e gerenciamento de memória.
Banco chave-valor com WAL	Meio	Durabilidade, recuperação após falha, índice em disco. Traz o Kleppmann para as mãos.
Clone das partes internas do git	Meio	Hash, objeto, árvore, commit. Git deixa de ser decoreba de comando.
Sistema completo em produção	Sempre	Pequeno, mas de verdade: deploy, monitoramento, backup, custo mensal conhecido. Vale mais que dez projetos de portfólio.
API idempotente com rate limiter	Meio	Chave de idempotência, token bucket, paginação por cursor, 429 com Retry-After. Pequeno, e ensina metade do que uma vaga sênior pergunta.
Fluxo OAuth 2.0 do zero	Meio	Authorization code com PKCE, dos dois lados. Depois disso você para de tratar identidade como caixa-preta.
Receptor de webhook confiável	Meio	Assinatura HMAC, proteção contra replay, retry com backoff, consumidor idempotente, dead letter queue. Um projeto, cinco conceitos difíceis.
Contribuição em open source	Sempre	Comece por documentação e bug pequeno. Ensina a ler código alheio e a receber revisão dura sem levar para o lado pessoal.

Os primeiros 90 dias

Começar é onde a maioria dos planos morre. Este é um recorte pequeno e executável do Horizonte 1, para tirar o roadmap do papel sem precisar decidir nada.

Período	Foco	Entregas concretas
Mês 1	Ferramentas e fundamentos	Missing Semester completo · OverTheWire Bandit até o nível 20 · primeiros capítulos do OSTEP
Mês 2	Dados e rede	Use The Index, Luke (completo) · otimizar a consulta mais lenta do seu trabalho · servidor HTTP do zero
Mês 3	Profundidade, segurança e entrega	Começar o Kleppmann (3 capítulos) · PortSwigger: trilhas de SQL injection, autenticação e controle de acesso · publicar o primeiro texto

Rotina semanal sugerida

- **Segunda a sexta, 1 hora:** 40 minutos de prática (código, laboratório, projeto) e 20 minutos de leitura.
- **Uma vez por semana, 30 minutos:** leia código que você não escreveu. Uma biblioteca que você usa, um pull request grande do seu time, um projeto open source.
- **Sábado, 1 hora, opcional:** escrever. Nota de estudo, ADR, postmortem, artigo. Publicar em algum lugar.
- **Domingo:** nada. Descanso faz parte do método, não é falha de disciplina.

REGRA DE DESISTÊNCIA

Se um recurso não estiver funcionando depois de duas semanas honestas, troque sem culpa. Livro abandonado não é fracasso, é informação: aquele autor não fala a sua língua. O que não pode é trocar toda semana — aí o problema não é o material.

Como saber que está funcionando

Horas estudadas é uma métrica ruim, porque mede esforço e não capacidade. Estes são sinais melhores:

- Você resolve sozinho um problema que há seis meses te faria pedir ajuda.
- Alguém da equipe procura você especificamente para uma classe de problema.
- Você discorda de uma decisão técnica e argumenta com trade-off, não com preferência.
- Você lê um código que a IA gerou e percebe, na hora, o que está errado nele.
- Você explica algo técnico para alguém não técnico e a pessoa entende.
- Você olha para código que escreveu há um ano e sabe exatamente o que faria diferente.

Uma última coisa

O fio que atravessa tudo neste documento é simples: a IA deixou barato produzir código e caro produzir certeza. Cada item aqui é, no fundo, uma forma de produzir certeza — sobre o que construir, se está correto, se vai aguentar carga, quanto vai custar. É nesse lugar que o seu valor vai morar, e é por isso que os fundamentos voltaram a importar mais do que a ferramenta da vez.

E nenhuma lista substitui o principal: construir coisas de verdade, colocar no ar, mantê-las funcionando e conviver com as consequências das próprias decisões. Todo o resto é apoio para isso.